TiffViewer コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

TiffViewerは、TIFFファイル用のビューアコンポーネントです。ブラウザの制限によりTIFF画像を直接表示できないため、ダウンロードを推奨するメッセージとダウンロード機能を提供します。

1.2 使用場面

- プロジェクト管理画面でTIFFファイルを選択した際の表示
- 実験結果として保存されたTIFF画像の閲覧・ダウンロード
- フローティングダイアログ内でのTIFFファイル表示

1.3 アクセス権限

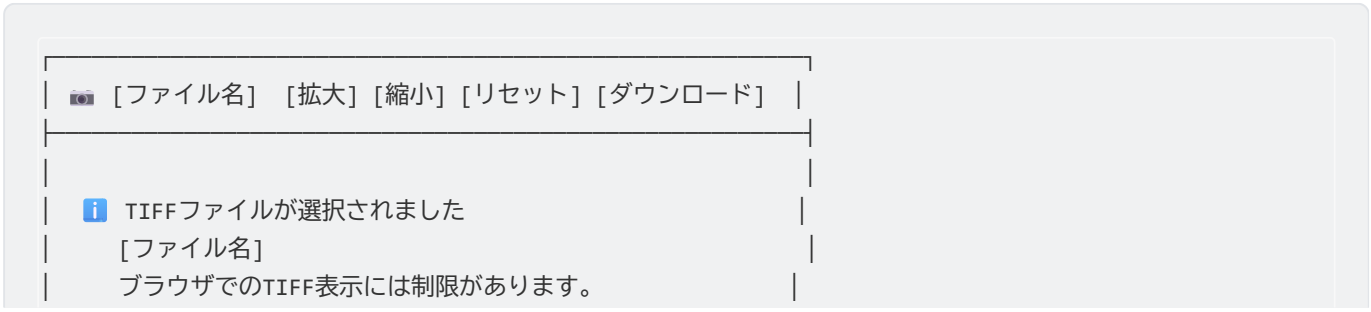
- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常表示時



```
│     ダウンロードボタンを使用して画像を表示することを          │
│     お勧めします。                                          │
│                                                             │
│  [TIFFファイルをダウンロード] (Primary Large)               │
│                                                             │
│  ┌─────────────────────────────────────────────┐           │
│  │ ファイルキー: ...                           │           │
│  │ ズーム倍率: 1.00x                           │           │
│  └─────────────────────────────────────────────┘           │
└─────────────────────────────────────────────────────────────┘
```

2.1.2 URLエラー時

```
┌─────────────────────────────────────────────────────────────┐
│ 📷 [ファイル名]  [拡大] [縮小] [リセット] [ダウンロード]  │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  ⚠️ URLの取得に失敗しました                                 │
│     [エラーメッセージ]                                      │
│                                                             │
│  ℹ️ TIFFファイルが選択されました                            │
│     （以下同じ）                                            │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 ツールバー

- **左側**: ファイルアイコン（FileImageOutlined、赤色 #CF0027）+ ファイル名（Title level 5）
- **右側**: 操作ボタン群
  - **拡大ボタン**: ZoomInOutlined アイコン（将来の機能拡張用）
  - **縮小ボタン**: ZoomOutOutlined アイコン（将来の機能拡張用）
  - **リセットボタン**: ReloadOutlined アイコン（将来の機能拡張用）
  - **ダウンロードボタン**: Primary、DownloadOutlined アイコン

2.2.2 情報表示エリア

- **背景色**: #f5f5f5（グレー）

- **配置**: 中央揃え
- **Infoアラート**: TIFFファイルのダウンロード推奨メッセージ
- **エラーアラート**: URL取得失敗時のエラーメッセージ（条件付き表示）
- **ダウンロードボタン**: Primary Large、署名付きURL取得後に有効化

2.2.3 ファイル情報カード

- **ファイルキー**: S3ファイルパス
- **ズーム倍率**: scale値（小数点2桁）※現在は表示のみ

---

3. 機能仕様

3.1 署名付きURLの取得

3.1.1 処理フロー

```
useEffect(() => {
  if (!fileKey) return;

  let isMounted = true;

  const fetchUrl = async () => {
    try {
      console.log('🔍 [TiffViewer] 署名付きURLを取得中:', fileKey);
      const url = await getFileViewUrl(fileKey);
      if (isMounted) {
        setSignedUrl(url);
        setUrlError(null);
        console.log('✅ [TiffViewer] URL取得成功');
      }
    } catch (err) {
      if (isMounted) {
        console.error('❌ [TiffViewer] URL取得エラー:', err);
        setUrlError(`URLの取得に失敗しました: ${err instanceof Error ? err.message : '不明なエラー'}`);
      }
    }
  };
```

```
  fetchUrl();

  return () => {
    isMounted = false;
  };
}, [fileKey]);
```

3.1.2 UI動作

- **開始時**: `getFileViewUrl(fileKey)` で署名付きURL取得
- **成功時**: signedUrlをStateに設定、ダウンロードボタンを有効化
- **エラー時**: urlErrorメッセージを設定、エラーAlert表示

3.2 ダウンロード機能

3.2.1 処理フロー

```
const handleDownload = useCallback((): void => {
  if (!signedUrl) {
    console.warn('[TiffViewer] ダウンロードURLがまだ取得されていません');
    return;
  }
  const link = document.createElement('a');
  link.href = signedUrl;
  link.download = fileName;
  link.click();
  console.log('✅ [TiffViewer] TIFFファイルダウンロード開始:', fileName);
}, [signedUrl, fileName]);
```

3.2.2 UI動作

- **ボタンクリック**: `<a>` タグを動的に生成してクリック
- **ダウンロード属性**: `download={fileName}` でファイル名を指定
- **署名付きURL**: S3の署名付きURLからファイルをダウンロード

3.3 ズーム操作（将来の機能拡張用）

3.3.1 拡大（Zoom In）

```
const handleZoomIn = useCallback((): void => {
  setScale(prevScale => Math.min(prevScale * 1.2, 5));
}, []);
```

- **倍率**: 現在の1.2倍
- **最大**: 5倍まで
- **現状**: scale値は保持されるが、表示には反映されない（将来の拡張用）

3.3.2 縮小（Zoom Out）

```
const handleZoomOut = useCallback((): void => {
  setScale(prevScale => Math.max(prevScale / 1.2, 0.1));
}, []);
```

- **倍率**: 現在の1/1.2倍
- **最小**: 0.1倍まで

3.3.3 リセット

```
const handleReset = useCallback((): void => {
  setScale(1);
}, []);
```

- **倍率**: 1倍（元のサイズ）に戻す

備考:

- ズーム機能は将来、TIFFをCanvas等で表示する際に使用予定
- 現在はscale値の管理のみで、実際の表示には影響しない

---

4. データ仕様

4.1 Props定義

```
interface TiffViewerProps {
  fileKey: string;
  fileName: string;
}
```

Props説明:

Prop名	型	必須	説明
fileKey	string	✅	S3ファイルキー（署名付きURL取得に使用）
fileName	string	✅	表示・ダウンロードするファイル名

4.2 State定義

```
const [scale, setScale] = useState(1);
const [signedUrl, setSignedUrl] = useState<string | null>(null);
const [urlError, setUrlError] = useState<string | null>(null);
```

State説明:

State名	型	初期値	説明
scale	number	1	ズーム倍率（現在は表示のみ、将来の拡張用）
signedUrl	string | null	null	署名付きURL（ダウンロード用）
urlError	string | null	null	URL取得エラーメッセージ

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
1	getFileViewUrl	-	-	✅ 実装済み	S3署名付きURL取得（fileAccessService）

5.2 API呼び出しコード

```
const url = await getFileViewUrl(fileKey);
```

備考:

- `getFileViewUrl` は `fileAccessService` で定義されたサービス関数
- バックエンドAPIを呼び出してS3署名付きURLを取得
- 有効期限付きURL（通常1時間）

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
署名付きURL取得失敗	urlErrorメッセージ設定、エラーAlert表示
ダウンロードボタン押下時にURLなし	コンソール警告出力、処理スキップ

6.2 エラー処理コード

6.2.1 URL取得エラー

```
try {
  const url = await getFileViewUrl(fileKey);
  if (isMounted) {
    setSignedUrl(url);
```

```
    setUrlError(null);
  }
} catch (err) {
  if (isMounted) {
    console.error('❌ [TiffViewer] URL取得エラー:', err);
    setUrlError(`URLの取得に失敗しました: ${err instanceof Error ? err.message : '不明なエラー'}`);
  }
}
```

6.2.2 ダウンロード時のURL未取得

```
if (!signedUrl) {
  console.warn('[TiffViewer] ダウンロードURLがまだ取得されていません');
  return;
}
```

エラーメッセージ:

- "URLの取得に失敗しました: [エラーメッセージ]"

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/TiffViewer.tsx`

7.2 関連ファイル

- **サービス**: `frontend/src/services/fileAccessService.ts`（getFileViewUrl）

7.3 依存コンポーネント

- なし（Ant Designコンポーネントのみ使用）

7.4 依存ライブラリ

- `antd` : Card, Button, Space, Typography, Alert, Flex
- `@ant-design/icons` : ZoomInOutlined, ZoomOutOutlined, ReloadOutlined, DownloadOutlined, FileImageOutlined

---

8. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline